



PROFESSOR: Diego Cardoso Borda Castro

CURSO: Sistema de Informação

DISCIPLINA: Programação Web

PERÍODO: 5º

DATA: 10/10/2025

ALUNO:

NOTA:

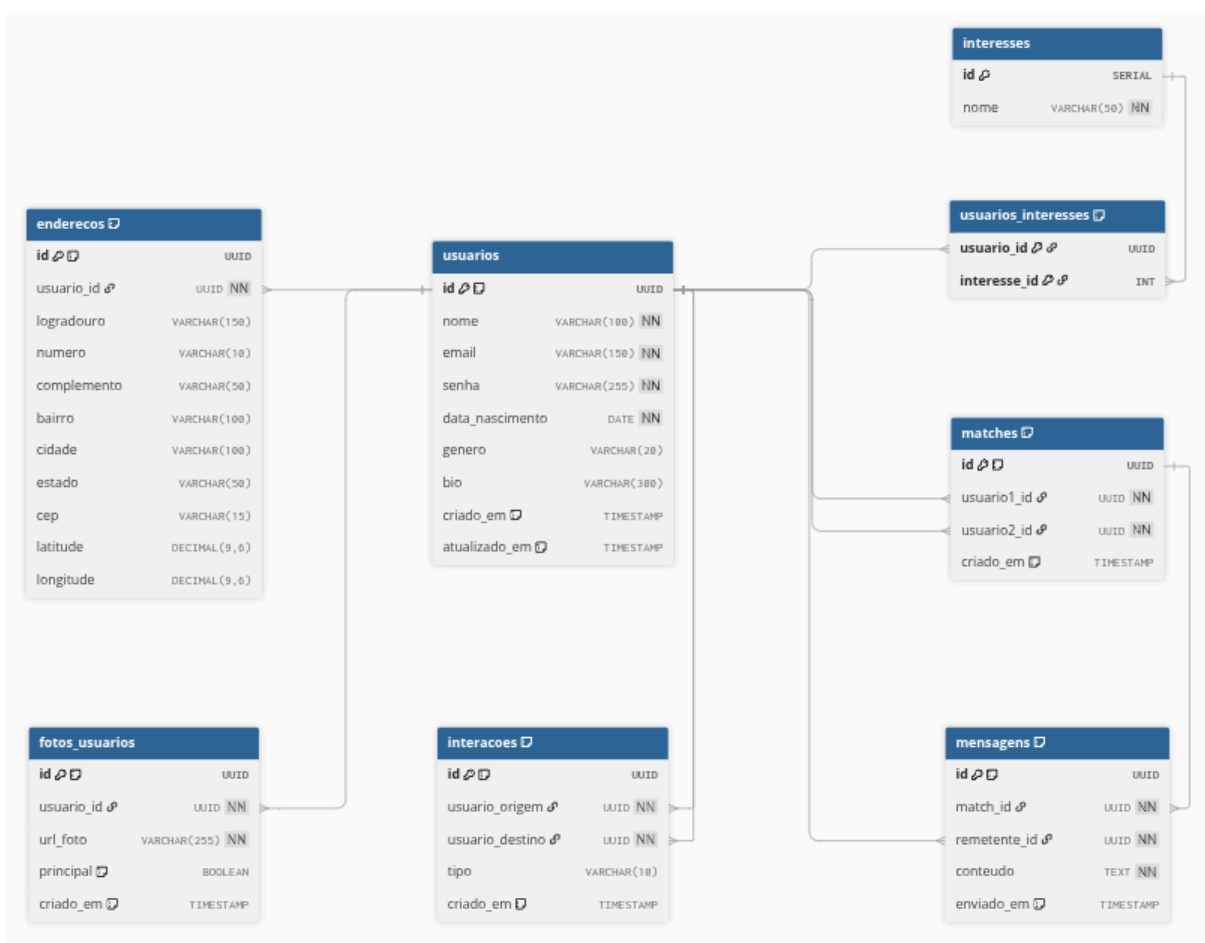
Você foi contratado para integrar uma equipe de desenvolvimento responsável por criar um aplicativo de namoro, semelhante ao Tinder. O sistema permitirá que usuários criem perfis, curtam outros usuários, façam “matches” (quando ambos se curtem) e troquem mensagens dentro do aplicativo.

Sua missão é demonstrar seu conhecimento e habilidades em Javascript ou Typescript, utilizando tecnologias como Express, Nest, Knex e Sequelize.

Considere que você faz parte de um time de desenvolvimento e as configurações de inicialização e conexões com o banco serão criadas por outro desenvolvedor. Portanto, não teremos que configurar package.json, tsconfig, nem configurar a conexão do Knex ou Sequelize.

O recorte do modelo do banco de dados a ser implementado pode ser visto a seguir:

- **Usuarios:** informações sobre os usuários cadastrados
- **FotosUsuarios:** armazenamento das fotos de cada usuário (relação 1:N com Usuarios);
- **Interacoes:** registros de curtidas e rejeições entre usuários
- **Matches:** registros de combinações bem-sucedidas
- **Mensagens:** mensagens trocadas entre usuários que deram match;
- **Interesses:** categorias ou tags que representam gostos pessoais (ex.: música, esportes, viagens);
- **UsuariosInteresses:** associação entre usuários e seus interesses (relação N:N);
- **Endereços:** informações geográficas para sugerir perfis próximos



Questão 1: Faça a inicialização do seu framework ou gerenciador de requisições, descrevendo a criação de uma rota para listagem de interesses de um determinado usuário. Não é para criar as camadas do endpoint, apenas definir a rota do mesmo **[0,5 ponto]**.

Questão 2: Implemente um CRUD para a entidade de Usuários, utilizando todas as camadas que foram apresentadas em sala de aula. Não é necessário acessar o banco de dados, mas a camada de acesso deve ser definida e utilizada, retornando apenas uma mensagem, ex: "Usuário criado!"

- i) Listar usuários: **/usuarios [GET] [0,5 ponto]**
- ii) Visualizar usuário: **/usuario/id [GET] [0,5 ponto]**
- iii) Criar usuário: **/usuario [POST] [0,5 ponto]**
- iv) Atualizar usuário: **/usuario/id [PUT] [0,5 ponto]**
- v) Deletar usuário: **/usuario/id [DELETE] [0,5 ponto]**

Questão 3: Crie um endpoint completo com todas as camadas necessárias que liste os usuários compatíveis com um determinado perfil, recebendo o ID do usuário logado e garantindo que nenhum usuário que já tenha tido uma interação anterior, seja curtida ou rejeição, com o usuário logado apareça na lista. Para isso, implemente os métodos necessários na camada de acesso ao banco de dados **[2 pontos]**.

Questão bônus: Explique a função das principais camadas e componentes do NestJS, descrevendo a responsabilidade de cada um e dando um exemplo prático de uso **[1 ponto]**.

- Modules
- Controllers
- Providers / Services
- Repositories (quando utilizados)
- Guards
- Interceptors
- Pipes
- Middleware
- Filters (Exception Filters)

Boa prova!